Project 2.1 — LoanDesk-Tool-Using-Finance-Assistant-With-A-Verifier

A practice twin of Project 2 — ResearchHand. Same skills — a tool-using **Worker** agent (a hand-built think→act→observe loop, three real tools, a hard step limit, clean recovery from a tool failure) paired with a separate **Verifier** agent that checks the answer against the tools' evidence before it goes out. A **new, real-world job**. **There are no steps here on purpose.** Read the story, build the whole thing yourself, and only open the hints or the solution if you get stuck.

Type: Multi-agent (2 agents: Worker + Verifier) · **Stack:** Python, LangChain (LCEL + `@tool`), a hand-built agent loop · **Level:** Beginner–Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

A customer walks up to a bank's front desk and asks: *"If I borrow 20,000 at 7% over 3 years, what's my monthly payment — and does my account's currency even matter?"* A clerk who answers from memory sounds confident and is often wrong. A clerk you'd trust **does the maths exactly**, **looks up the live exchange rate** instead of guessing it, **checks the policy** instead of winging it — and for anything that touches money, has a **second person sign off** before the answer leaves the desk.

You are going to build **LoanDesk**: that trustworthy clerk, as two agents.

- The **Worker** is the clerk who looks things up. Given a question, it runs a loop and calls real tools instead of guessing: an exact **loan-payment calculator**, a **live exchange-rate** lookup, and a **policy search**. It works one step at a time, with a hard limit so it can never spin forever.
- The **Verifier** is the second person. It reads the question, the Worker's answer, and **what the tools actually returned**, and asks "does this really hold up?" If the answer isn't grounded in the tool evidence, it says so — and the Worker gets exactly one chance to fix it.

The one lesson of this project: a model states a wrong number as confidently as a right one. When the number is someone's loan payment, "confident" isn't good enough — you need an agent that *acts* (looks up, calculates) and a second one that *checks*.

How it works, from the question to the answer (picture the whole machine)

Read this once and the whole project should form in your head.

You type a question. The **Worker** binds its three tools to the model and enters a loop. Each turn it asks the model what to do; if the model asks for a tool, the Worker runs it, feeds the result back as an observation, and loops again. For *"monthly payment on 20,000 at 7% for 36 months"* the model calls the **payment calculator** (exact formula, not a guess). For *"what's that in dollars?"* it calls the **exchange-rate** tool (today's real rate, not its training data). For *"is early repayment allowed?"* it calls the **policy search**. The Worker keeps a **step log** of every call, and a **hard step limit** means it stops even if the model keeps asking — raising a clear error with the partial log instead of looping forever.

One of the tools **fails on its first call on purpose** (networks do that). The Worker doesn't crash: the failure comes back as a plain `Error: …` observation the model can read, so it simply **tries again** — proving the loop survives a real-world flake.

When the Worker stops, the **Verifier** takes over. It gets the question, the Worker's answer, and **only the successful tool results**, and returns a checked verdict: `approved: true/false` + a reason. If it rejects the answer, LoanDesk adds the reason to the task and runs the Worker **once** more, then verifies again. The user only ever sees an answer that a *separate* agent — with its own prompt and its own blind spots — signed off on.

That's the whole machine: **Worker loops over real tools (with a limit and failure recovery) → Verifier checks the answer against the evidence → one retry if rejected → only then does it reach the user.**

What you are building (the pieces)

- A plain **LCEL chain** that answers from memory (to *show* why guessing isn't enough) — the baseline the compare script contrasts with.
- A **Worker**: `bind_tools`, a hand-built loop (`find_tool`, `run_tool` returning `(result, error)`, an `AgentResult` step log), a hard `max_iterations`, and `MaxIterationsExceeded` carrying the partial log.
- **Three tools**: `monthly_payment` (exact amortization math — pure logic), `get_exchange_rate` (a live, keyless API), and `search_policies` (fails once, then works).
- A **Verifier**: `with_structured_output(VerifierVerdict)`, fed only the successful tool evidence.
- A **service** that runs Worker → Verifier → one retry on rejection.

Inputs: a finance question. **Outputs:** a step log plus an answer marked verified / NOT verified. **Rules:** the Worker only answers from tool results; the loop always stops; a rejected answer gets exactly one retry.

A real example to build and test against

- *"What's the monthly payment on a 20,000 loan at 7% annual for 36 months?"* → `monthly_payment` → exact number; Verifier approves.
- *"What is 500 EUR in USD today?"* → `get_exchange_rate` → live rate; Verifier approves against the tool's result.
- *"Does our policy allow early repayment without a fee?"* → `search_policies` (fails once, retried) → policy text; Verifier approves.
- A deliberately wrong answer in a test → the Verifier returns `approved: false` with a reason, instead of letting it through.

How to practice this

1. Read this story. **Don't open the solution yet.**
2. Plan it on paper: the three tools, the loop, the step limit, failure recovery, and the Verifier handoff.
3. Build it yourself. A good order: a plain chain that only guesses → one tool + a hand-built loop → all three tools with a limit and recovery → the Verifier and the one-retry handoff. Your call.
4. Test it on the real example: a calculation, a live lookup, a policy search, and a wrong answer the Verifier should catch.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

This is the shape of every assistant that must answer with **live or exact information** — a price, a rate, a balance, a policy, a calculation — instead of a memorized guess, and then **check itself** before acting. Bank helpdesks, insurance quoting, billing support, internal finance bots: all of them are a tool-using worker plus an independent checker. The hand-built loop is the building block of every bigger agent you'll meet later (supervisors, RAG agents, multi-agent teams) — building it yourself once means you understand it, not just call it. And the "act, then verify against the evidence" habit is what separates a chatbot from a tool a business would actually trust with money.

Done when

- A calculation uses the calculator; a currency question uses the live rate; a policy question uses the search — and the step log shows it.
- The failing tool is recovered from (retried), not crashed on; the loop always stops (a hard limit raises `MaxIterationsExceeded` with the partial log).
- The Verifier approves a right answer and rejects a deliberately wrong one; a rejected answer gets exactly one retry.
- Your tests pass: the settings, each tool called directly (no network/model), the Worker's helpers, and a live run of the Worker + Verifier.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.